Exploration, not Value Estimation, Limits Learning Agents on ARC-AGI-3

[NAME 1][1], [NAME 2][1]

[NAME 1: student number, student email]; [NAME 2: student number, student email]

[1]**School of Computer Science and Applied Mathematics, University of the Witwatersrand, Johannesburg**

Team *Vladmir_Putin*, COMS4061A/7071A Reinforcement Learning, 2026

Abstract

ARC-AGI-3 asks an agent to discover the rules of unseen games from pixels and to finish their levels in as few actions as a human. We study what limits learning agents here. DQN (with test-time TD learning) and PPO, trained offline on 15 games, each get two stacked improvements, and every improvement is motivated by a measured failure of the previous stage. On a frozen 5-game dev split (500 actions/game, 3 training $\times$ 3 evaluation seeds) both baselines are indistinguishable from random (official score 0.0000 and 0.0004 vs. 0.0004). The best learned agent, PPO with random network distillation, reaches 0.025. All learned agents waste their budget re-trying actions that change nothing, and pretrained effect predictors are at chance on unseen games. Replacing learned estimates by explicit per-game online knowledge (a state graph with Thompson-sampled action effects) raises dev score to 0.79, but these gains overfit the dev games: a version with dev score 0.50 scores 0.07 on the hidden Kaggle games. Our leaderboard entry is therefore an attributed fork of Franzen's public LLM agent with three of our own fixes (29.15). We show that single Kaggle scores in this regime are dominated by run-to-run variance and run failures, and we report every run. Code: https://github.com/ThendoNelufule/rl-arc-agi3.

1 Problem and research question

ARC-AGI-3 (ARC Prize Foundation, 2026; Chollet, 2019) is a benchmark of turn-based 64$\times$64 games with no instructions: the agent must infer controls, goals and rules by acting, and is scored by how efficiently, relative to humans, it completes levels. The ARC Prize 2026 Kaggle competition (ARC Prize Foundation and Kaggle, 2026) evaluates agents offline on 110 hidden games within 9 hours. Rewards are sparse (a random agent completes about one level per 7,500 actions on our training games) and each game is new, so knowledge transfers only at the level of general behaviour.

Research question. *Can a value-based (DQN) or a policy-gradient (PPO) agent trained offline on public ARC-AGI-3 games learn exploration behaviour that completes levels in unseen games, and what limits it?* **Contributions.** (i) A Gymnasium adapter and a fixed protocol with a frozen train/dev/held-out split. (ii) Two evidence-driven chains (DQN: novelty, effect masking; PPO: RND, recurrence), all runs reported, with a diagnostic explaining each failure. (iii) A graph explorer built from these diagnostics, including its generalisation failure. (iv) A variance analysis of Kaggle scores for the LLM agent we forked, which we report separately because it is mostly other people's work.

2 Background and prior work

Scoring. ARC-AGI-3 uses relative human action efficiency (RHAE): a completed level l with human baseline b_l actions, solved in a_l actions, earns $100\min(1.15, (b_l/a_l)^2)$. Unfinished levels earn 0. A game's score is the level-index-weighted mean ($w_l = l$), and the benchmark score is the mean over games. Late levels therefore dominate, and speed on level 1 is worth little. **DQN** (Mnih et al., 2015) learns $Q_\theta(s,a)$ by minimising the Huber loss to $y = r + \gamma \max_{a'} Q_{\bar\theta}(s',a')$, using replay and a target network. **PPO** (Schulman et al., 2017) maximises the clipped surrogate $L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon)\hat{A}_t)]$, where $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$. It uses GAE advantages $\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}$ with $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (Schulman et al., 2016), a value loss and an entropy bonus. Invalid actions are masked in the softmax (Huang & Ontañón, 2022). **Exploration bonuses:** count-based novelty (Bellemare et al., 2016) and random network distillation (RND), the prediction error of a trained network against a fixed random one (Burda et al., 2019). **ARC-AGI-3 agents.** *Stochastic Goose* (Smit, 2025), the preview winner, trains a CNN online to predict which actions change the frame; our effect masking and graph explorer build on this idea, and return-to-frontier search relates to Go-Explore (Ecoffet et al., 2021). The 2026 leaders are LLM tool agents. Franzen's Milestone-2 solution (Franzen, 2026) patches Tufa Labs' *Duck* harness (Cottaar & Tufa Labs, 2026), the Milestone-1 winner, adding tools, guards and a scheduler over 110 games, with SGLang (Zheng et al., 2024) serving a 4-bit Qwen3.8-Flash-Next (Qwen Team, 2026). OpenAI reported that retaining reasoning between turns, plus compaction, tripled their score (OpenAI, 2026).

3 MDP formulation

Each game is an episodic POMDP (hidden counters, §6.6), which we treat as an MDP over observed frames. Our adapter (`rl/arc_gym.py`) defines the following. **Observation** $o_t \in \{0..15\}^{1\times64\times64}$: the final animation layer as colour ids, one-hot encoded inside the network. Raw pixels keep the formulation game-agnostic. **Actions** `Discrete(70)`: 0–4 = ACTION1–5, 5 = ACTION7 (undo), and 6–69 = ACTION6 clicks at the centres of an 8×8 grid of 8×8-pixel cells. A single categorical space with a spatial click head keeps clicks learnable in 400k steps. Illegal actions are no-ops that still cost a step, and `action_masks()` exposes legality. Only the graph explorer may also click any pixel (index $70 + 64y + x$), a change approved after the grid was shown to be its bottleneck (§6.3). **Reward** $r_t = \max(0, \Delta\,\text{levels_completed})$, used for training only; we always report the official score. **Termination** on WIN or GAME_OVER, and truncation at 500 actions in training. In evaluation, GAME_OVER triggers the engine's reset and play continues within the same 500-action budget. **What persists.** Between actions: DQN-online weights and its replay of this game, novelty counts, the graph and its statistics, and the LSTM state. Between levels: weights and per-game statistics persist, but the graph is cleared. Between games and evaluation runs: nothing. Weights are restored to the pretrained checkpoint, and each seed gets a fresh Arcade and scorecard. **Offline training vs. online adaptation.** Learning from other games happens offline on the 15 training games. On an evaluation game, adaptation happens only through the agent's own interaction: test-time TD (DQN-online), recurrent state, or explicit memory (graph). No game-specific code or hidden engine state is used.

4 Methods

All learned agents share one encoder (strided convolutions to an 8×8 map aligned with the click cells; a spatial 1×1-convolution click head and an MLP head for simple actions; 1.19M parameters for DQN), the same 15 training games run as 15 lock-step environments, 400k environment steps, and an episode cap of 500 (full hyperparameters: Table 3, supplement).

DQN chain. T10 (baseline, pre-registered): DQN (Mnih et al., 2015) with masked targets. At evaluation (`dqn-online`) it takes one TD update per action on the game being played (ϵ=0.05), with weights reset to the pretrained ones at the start of every game; the frozen network is reported

as variant T10f. T11 adds a count-based novelty bonus $\beta/\sqrt{N(s')}$ (β=0.05, exact frame counts; episodic in training, per game at evaluation). T12 adds an action-effect head trained with BCE on "frame changed". Actions with predicted $p(\text{change}) < 0.1$ are masked, and at evaluation an exact per-game no-op memory is kept and the head is fine-tuned online.

PPO chain. T20 (baseline, pre-registered): MaskablePPO (Raffin et al., 2021; Huang & Ontañón, 2022). *What is learned:* a masked actor $\pi_\theta(a|s)$ and a critic $V_\phi(s)$ on the shared encoder. *When:* after every rollout of 128 steps $\times$ 15 environments (1,920 transitions), GAE (γ=0.99, λ=0.95) advantages are computed and θ, ϕ are updated for 4 epochs of minibatches of 240 on L^{CLIP} (ϵ=0.2) $- c_v L^V + 0.01\,\mathcal{H}[\pi]$. *How it affects decisions:* at evaluation the weights are frozen and actions are *sampled* from the masked policy. An on-policy update inside a 500-action budget would be only a few small batches, so the PPO chain adapts through memory instead. T21 adds RND (Burda et al., 2019) with coefficient 0.05 (one combined reward stream). First tests showed the bonus collapsing to 10^{-4} within 4k steps, because ARC frames are mostly identical background. Orthogonal target initialisation, per-dimension standardisation of target features, and normalising the bonus by an EMA of the mean error keep it at 0.02–0.05 per step. T22 makes the T21 policy recurrent (masked RecurrentPPO, LSTM 256 (Hochreiter & Schmidhuber, 1997)), with the hidden state reset per game and after GAME_OVER.

Graph explorer (cross-chain extension, `rl/policies.py`). It keeps an exact frame-hash state graph per level and never re-tries recorded no-op or deadly (s, a) pairs. It walks a BFS path to the nearest state with untried actions and ranks untried actions by a per-game Beta-Thompson (Thompson, 1933) estimate of their effect, keyed by action or by the colour under the click. T30 uses no weights at all. T31 adds $0.3\,\pi(a|s)/\max\pi$ from T21 as a prior. Later versions (T32–T47) each fix one failure visible in recordings: step-counter ("ticker") rows detected online and masked, pruning of actions proven ineffective, visual click equivalence, learned inverse-action pairs (T37), object clicks (T41–T44), and one click per connected component (T45).

Final leaderboard entry. Franzen's public Milestone-2 notebook (Franzen, 2026) on Tufa Labs' Duck harness (Cottaar & Tufa Labs, 2026): the agent, model, serving and scheduler are theirs. **Our changes** (MOD-F) are three fixes from our own log audit. (1) Reasoning-only replies cut off at the 12,288-token output cap are replaced by the harness's existing stub; 13 of 2,341 replies in our proxy re-sent ∼13k tokens of broken reasoning. (2) A false sentence in the stale-state refusal is corrected and one prompt line states the no-op rules; 92 actions were silently refused after the old advice. (3) The per-game cap is 525 instead of 532 min, because proxy timings projected 32,524 s against a 32,400 s limit. We also tested MOD-1 (two reused public add-ons (heliosli, 2026; shiiin9 & Affectify ARC, Osaka Metropolitan University, 2026)) and MOD-2 (MOD-1 + Fable facts (leoprovorov, 2026) + our reinvestment decay).

5 Experimental setup

Environments. We used the 25 public ARC-AGI-3 games in offline mode (`arc-agi` 0.9.8, `arcengine` 0.9.3) and no others. They were split once, stratified by type (move/click/mixed), and frozen (`splits.json`, seed 2026): *train* (15: ar25, cn04, ft09, g50t, m0r0, r11l, re86, s5i5, sb26, sc25, sp80, su15, tr87, vc33, wa30), *dev* (5: dc22, ka59, lf52, lp85, ls20; all decisions), and *held-out* (5: bp35, cd82, sk48, tn36, tu93; evaluated once after development ended). Kaggle uses hidden games. The LLM-agent proxies use all 25 public games, because that agent has no training split. **Protocol** (`rl/evaluate.py`, identical for every agent). The budget is 500 actions per game, counted across deaths, and play stops early only on WIN. Each seed gets a fresh Arcade and scorecard. We log levels, actions, deaths, level-vs-action curves, recordings and the official score for every run. Learned agents use 3 training $\times$ 3 evaluation seeds, and we report the mean $\pm$ SD over training seeds plus all 9 runs. Training-free T30/T35n/T45n use 9 evaluation seeds. **Budgets, stopping, checkpoints.** A fixed 400k steps, with no early stopping, and the final checkpoint is used. Training seed 0 for Kaggle, the baselines, and the evaluation-time convention were all pre-registered before any result. **Hardware.** Training ran on Kaggle T4 GPUs (4 vCPU Xeon 2.0 GHz) and eval-

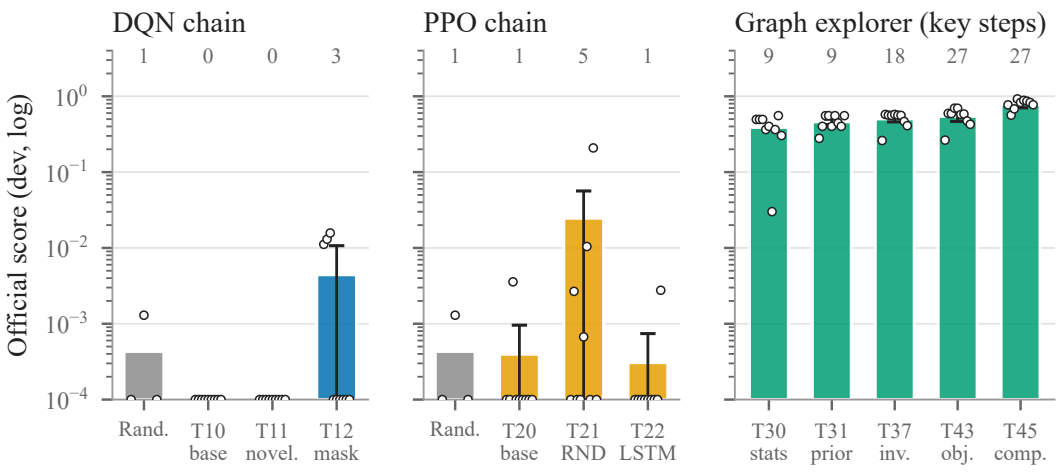


Figure 1: Official dev score (log scale; numbers = levels over all runs). Bars: mean over 3 training seeds, error bar: SD over training seeds; dots: all 9 runs, zeros at 10^{-4}. Random/T30: no training (3/9 eval seeds). Colour = agent family throughout.

Table 1: Master results (official score). Dev: mean $\pm$ SD over training seeds; Lv: levels over all dev runs (of 342; random of 114); level 1: solving runs @ actions taken. No agent finished a game, so every run used all 500 actions. Train/held-out: means over 3–9 eval seeds (learned: training seed 0); –: not run. Kaggle: public LB, training seed 0. †Starter random agent.

Ver.	Change	Dev score	Lv	Level 1 on dev: runs @ actions	Train	Held-out	Kaggle
Rand.		0.0004 ± 0.0006	1	lp85 1/3 @352	0.0111	–	0.02†
T10	DQN	0	0	–	–	–	0.00
T11	+novelty	0	0	–	–	–	0.00
T12	+effect mask	0.0044 ± 0.0063	3	lp85 3/9 @101–120	–	–	0.00
T20	PPO	0.0004 ± 0.0006	1	lp85 1/9 @212	–	–	0.00
T21	+RND	0.0247 ± 0.0317	5	lp85 4/9 @124–489; ls20 1/9 @41	–	–	0.05
T22	+LSTM	0.0003 ± 0.0004	1	lp85 1/9 @241	–	–	–
T30	graph+online stats	0.389 ± 0.149	9	lp85 9/9 @16–72	0.0578	–	–
T31	+PPO prior	0.461 ± 0.038	9	lp85 9/9 @12–24	–	–	–
T37	+inverse actions	0.504 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	0.07
T43	objects as fallback	0.545 ± 0.079	27	lf52 9/9 @52–349; lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	–
T45	component clicks	0.791 ± 0.087	27	lf52 9/9 @24–56; lp85 9/9 @14–30; ls20 9/9 @128–287	–	–	–
T45n	no prior	0.667 ± 0.222	28	lf52 9/9 @23–59; lp85 9/9 @11–33; ls20 8/9 @128–374	0.0811	–	–

uation on Kaggle CPU kernels (1 thread per job; after a laptop out-of-memory crash, three agents were re-run there with identical results). Python 3.13.15, PyTorch 2.11.0, NumPy 2.1.3, Gymnasium 1.2.0 (Towers et al., 2024), SB3/sb3-contrib 2.9.0; the submission image uses Python 3.12.13 and PyTorch 2.10.0 (CPU). **Cost** (Table 2, supplement). *Training:* 0.30–0.50 T4-hours per seed for DQN versions, 0.37–0.46 for PPO and 0.94 for PPO-LSTM; 10.45 T4-hours in total over 21 runs. *Evaluation* (3 seeds $\times$ 5 games $\times$ 500 actions, 1 CPU thread): 1,066–1,350 s for DQN-online (test-time TD), 68–110 s for PPO, and 89–123 s for the graph explorer, which needs no training beyond the T21 prior.

6 Results

6.1 Baselines: both learners are indistinguishable from random

Random scores 0.0004 on dev (1 level in 3 runs), T10 and its frozen variant score 0 (0 levels in 9), and T20 scores 0.0004 (1 in 9). On Kaggle all three score 0.00–0.02 (Table 1). Fig. 2 explains why. In 400k steps the DQN seeds see only 7/4/8 rewards, the Q-values stay flat ($\approx$0.05, loss $\approx 10^{-5}$),

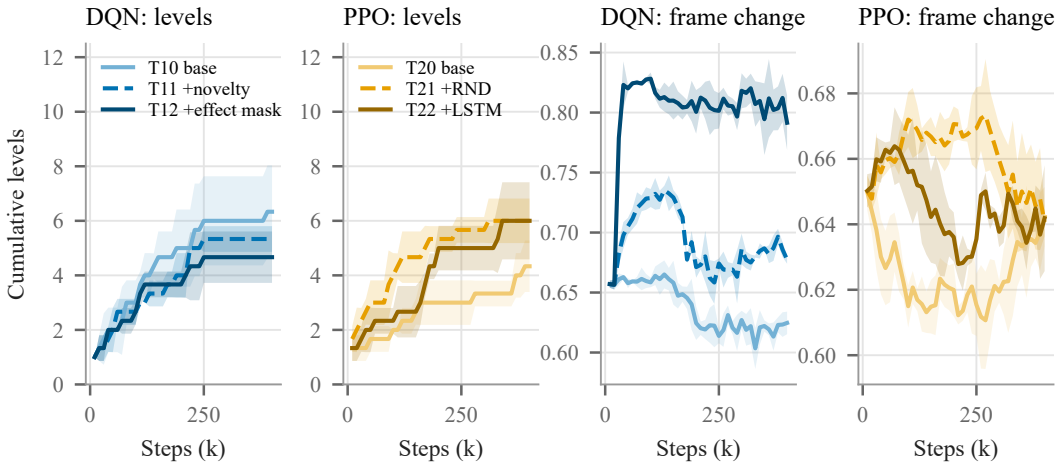


Figure 2: Training curves, mean ± SD over 3 training seeds. Left: cumulative level completions (all rewards seen in 400k steps on 15 games). Right: fraction of actions that change the frame. Effect masking (T12) raises the frame-change rate from 0.61 to 0.77–0.82 without increasing reward, and no version gets more than 8 rewards per run.

and greedy selection degenerates: the frozen T10f spends 77–93% of its actions on one action, and with test-time TD the top-action share is still 0.34–0.40 (random: 0.08). PPO does not collapse, but with 3–5 rewards per seed its entropy only falls from 3.49 to ≈3.0 nats, leaving near-uniform behaviour. *The limit is the missing learning signal, not capacity.*

6.2 Improvements: each stage removes one diagnosed failure

Each step below runs limitation → hypothesis → intervention → result. **T11 (novelty).** *Limitation:* no learning signal (T10). *Hypothesis:* a dense count bonus breaks the collapse. *Result:* the training frame-change rate rises from 0.61 to 0.67, but dev score stays 0 and the collapse is slightly worse (top action 0.50–0.58). New diagnostic: 27–33% of *all* actions repeat an (s, a) already seen to change nothing (random: 7%), which the bonus penalises only slowly through TD. **T12 (effect masking).** *Hypothesis:* predicting and masking ineffective actions removes this waste. *Result:* repeated known no-ops fall to 0.169/0.012/0.042, and T12 is the first DQN version to complete levels on dev (0.0044 ± 0.0063, 3 levels, all lp85; frozen variant 0.0036, 6 levels). However, the pretrained effect head has a within-game AUC of 0.42–0.58 on unseen dev games: *chance*. A PPO-trained effect model (T23) is also at chance (0.39–0.54, excluding the trivial ls20). The gain comes from the exact per-game memory, not from pretraining. **T21 (RND).** *Limitation:* PPO barely moves from uniform. *Hypothesis:* a learned novelty signal that generalises across near-identical frames gives a dense gradient. *Result:* the largest learned-agent gain, 0.0247 ± 0.0317 with 5 levels (lp85 4/9, ls20 1/9). The variance is large: one ls20 solve in 41 actions scores 0.2085 by itself and dominates the mean, because efficiency is squared. **T22 (LSTM).** *Limitation:* the memoryless T21 still re-tries known no-ops (8–10% of actions). *Hypothesis:* recurrence lets the policy remember failed actions. *Result:* **failure**, 0.0003 with 1 level. Repeated no-ops are not reduced (0.059–0.138), and training cost doubles (0.94 h/seed). 400k steps on 15 games do not teach an LSTM to "not repeat what failed". *Established:* for both algorithms the constraint is exploration inside an unseen game. The knowledge that helps (what does nothing, where the agent has been) is game-specific and must be explicit and acquired online.

6.3 Graph explorer and ablations

The graph explorer applies these findings directly. With online statistics alone, T30 scores 0.389 and solves lp85 level 1 in every run. The T21 policy as a prior gives T31 0.461 ± 0.038. Each later

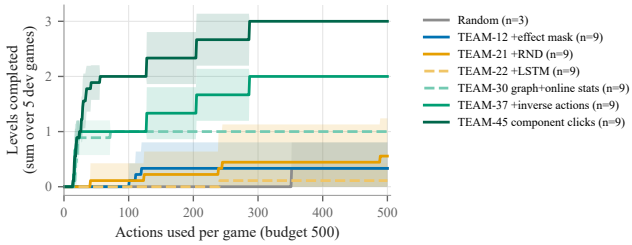


Figure 3: Levels completed on the 5 dev games vs. actions used (mean ± SD over runs). Learned agents complete levels late and rarely. The graph explorer solves lp85 level 1 in 14–30 actions (human baseline 17) and lf52 level 1 in 24–56 (human 32), but no version finishes a game or reaches level 2 on lp85/ls20 within 500 actions.

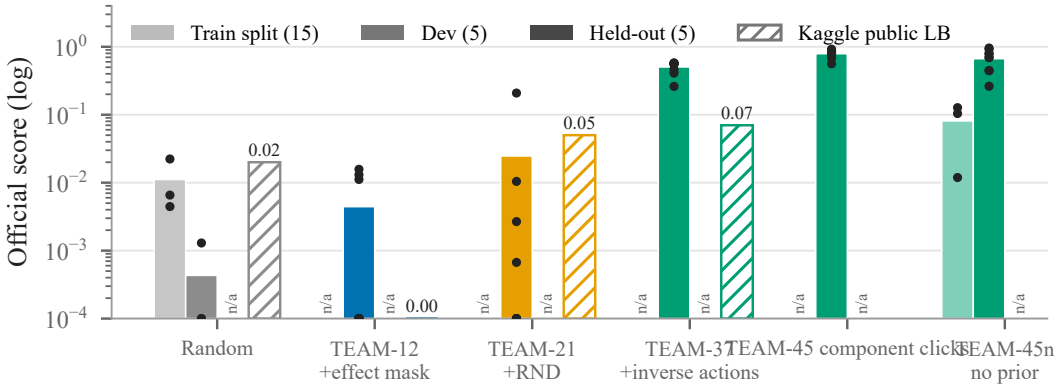


Figure 4: Official score by evaluation set (log scale; dots = runs). Train: training seed 0 (for learned agents, their own training games). Held-out: evaluated once, after development. Kaggle: one public-LB run on hidden games.

step was diagnosed from recordings (supplement, Fig. 7). Masking the step-counter ("ticker") rows restores state identity, because the bar otherwise makes every frame new (T32). Inverse action pairs learned online cut navigation cost: ls20 level 1 is solved in 9/9 runs after 128–287 actions instead of 6/9 after 367–473 (T37, 0.504, 18 levels). Every remaining failure traced to the 8×8 click grid: with 1000 or 2000 actions T37 reaches the same 18 levels, because its reachable frontier is exhausted. With pixel clicks, objects as a fallback tier (T43, 0.545, 27 levels) and one click per connected component (T45, $\mathbf{0.791 \pm 0.087}$; lf52 level 1 at human speed) gave the largest gain. Some variants failed: patch-keyed statistics (T38, 0.124) slowed lp85, and a size prior (T47, 0.632) traded lp85 for lf52. **PPO prior ablation.** Without the prior, T35n scores 0.471 with 15 levels, vs. T35 0.493 with 9. The prior speeds up lp85 clicks but biases movement and prevents ls20 solves, and applying it to clicks only (T36: 0.494, 15 levels) keeps both gains. For the final design, T45n scores 0.667 vs. T45 0.791. The difference comes from ls20 (8/9 vs. 9/9 solves, up to 374 vs. 287 actions) and is within one SD of T45n's runs (0.222). The learned prior contributes a small, game-dependent ordering bias. The explorer's structure, not learning, produces the gain.

6.4 Generalisation: dev gains do not transfer to hidden games

The graph explorer's dev score was optimised over 18 versions on 5 games, and it shrinks off dev. On the 15 training games, which it never saw, the no-prior logic scores 0.121 (T35n; dev 0.471) and 0.081 (T45n; dev 0.667). On Kaggle, T37 (dev 0.504) scores **0.07**. Held-out: T45 pending and T37 pending (random pending). *Dev overfitting is the main lesson of the graph track.* Five dev games with one or two solvable first levels rewarded orderings fitted to them, and the late levels that RHAE weights most were never reached.

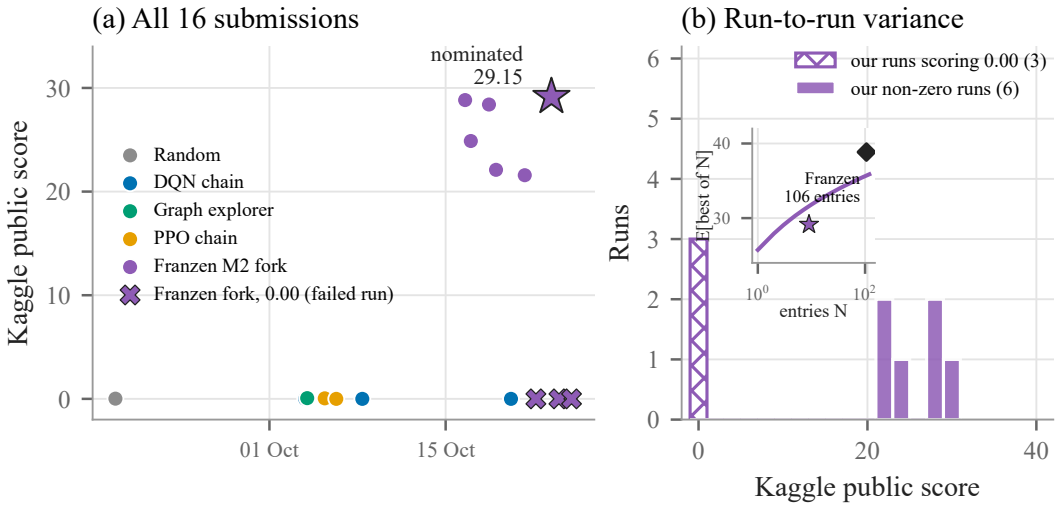


Figure 5: (a) All 16 of our Kaggle submissions (public score) by date. $\times$: Franzen-fork runs scoring 0.00 (byte-identical 3,411 B outputs); $\star$: nominated entry. (b) Our 9 Franzen-family runs. Inset: expected best of N under $\mathcal{N}(25.77, 3.93^2)$ (public copies of the notebook), with Franzen's best (38.85 from 106 entries) and ours (29.15 after 9).

6.5 Final leaderboard entry and the variance of Kaggle scores

Attribution. The entry is Franzen's agent (Franzen, 2026) on Tufa Labs' harness (Cottaar & Tufa Labs, 2026). Our share is the selection, packaging, analysis, and the three MOD-F fixes. **Results.** Base 28.83; MOD-1 24.88 and 21.58; MOD-F 28.40, 22.10, 0.00, **29.15**, 0.00, 0.00 (Fig. 5a). We nominate MOD-F submission 57556604 (29.15, 2026-10-23 09:42:37 UTC; rank 959 of 4,817 on 2026-10-25). Its mean over 6 runs is only 13.28 (26.55 without the zeros). **What a single score can show.** Public copies of this notebook score 25.77 ± 3.93. A leaderboard entry is a team's *best* submission, so under that spread the expected best of N grows from 25.8 (N=1) to ≈35.6 (N=100). Franzen's 38.85 from 106 entries is consistent with this plus genuine improvements (Fig. 5b). Our base, MOD-1 and the non-zero MOD-F runs all lie inside the spread: *we cannot claim that any modification changes the expected score*. On the 25-game proxy, MOD-2 minus MOD-1 was -4.56 (95% bootstrap CI -15.2 to $+5.4$; per-game SD ≈26), so one pass resolves only effects of about ± 10. **Honest failures.** *MOD-2* missed its pre-registered threshold ($\geq$+2). Its logs show seven games that stopped acting 18–69 min into the run, starved by our reinvestment decay (priority $\times 0.2$ after 130k tokens on a level). Five of them explain ≈7.8 points of the drop: a design error in our add-on. *Retained reasoning* (OpenAI, 2026) is already on in the base. Prior thinking is re-sent every turn (prompt growth per generated token: median 1.83; prefix-cache hit rate 93%), so we built no MOD-3. *MOD-F zeros:* three runs produced byte-identical outputs in which no game scored, i.e. whole-run failures. Our changes are deterministic, the same version scored three times, and the inputs were unchanged. If zeros fell at random among our 9 Franzen-family runs, all 3 would land in MOD-F's 6 with probability 0.24. We find no evidence that our fixes caused them, but the rerun logs are not visible to us, so the cause is unknown.

6.6 Failure cases

On move and mixed dev games every agent dies on a fixed per-life step budget (lives of ∼129/125/100/65 steps on ls20/dc22/ka59/lf52, identical for random, PPO and graph agents), so banning the last action before a death is wrong (T34). On dc22 and ka59 the reachable states are exhausted within a few hundred actions (Fig. 6), and relaxing pruning (T40) finds no new ones. What is missing is a hypothesis about the goal, not more coverage. The LLM agent shows the same

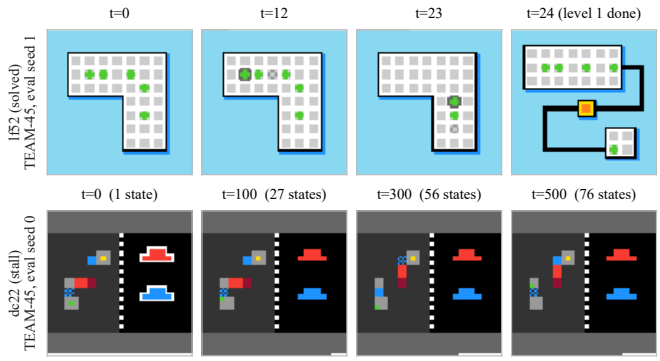


Figure 6: Recorded T45 frames. Top: lf52 level 1 solved in 24 actions (human 32); the next level's layout appears at $t{=}24$. Bottom: dc22 stall, with only 76 distinct ticker-masked states after 500 actions.

pattern: in Franzen's 100 public trajectories, stalls on later levels cost 43.9 points but inefficiency on solved levels only 2.6 (skarin, 2026).

7 Discussion and limitations

Established. (1) With sparse level rewards and 15 training games, neither DQN nor PPO learns transferable exploration, even with count, RND or effect shaping. Gains appear only with explicit per-game memory. (2) Pretrained effect predictors are at chance on unseen games, so "what changes the frame" must be learned online. (3) Hand-designed structure is very sample-efficient on the games it is tuned on and generalises poorly. **Uncertain.** Graph versions within $\approx$0.05 of each other are within seed noise (SD 0.04–0.09). Dev and held-out have 5 games each. Kaggle scores are single runs with $\pm$4 noise and unexplained failures. Compute is not matched to the LLM agent (an RTX PRO 6000 for 9 h, vs. CPU seconds for ours). **Limitations.** 400k steps is small for deep RL (Henderson et al., 2018), although 3–8 rewards per run suggests that the signal, not the budget, binds. The Kaggle checkpoint (training seed 0) was pre-registered, not chosen as the best seed.

8 Conclusion

For learning agents on ARC-AGI-3 the bottleneck is exploration within an unseen game, not value estimation or policy optimisation. Every gain we measured came from making per-game knowledge explicit and online, and those gains overfit a small dev set. A competitive score required a large pretrained model, whose single-run scores are too noisy to rank small modifications. Learning the explore-then-hypothesise loop itself, rather than hand-coding it, remains the open problem.

Generative-AI declaration. Per the Wits academic-integrity policy: we used Claude and Claude Code (Anthropic) for planning, code, data analysis and drafting. We verified all code, numbers and text against our logs and take full responsibility for the content.

Member contributions. [NAME 1]: [to be completed by the team]. [NAME 2]: [to be completed by the team].

References

ARC Prize Foundation. ARC-AGI-3: Interactive reasoning benchmark. https://arcprize.org/arc-agi/3, 2026. Accessed 2026-10-25.

ARC Prize Foundation and Kaggle. ARC Prize 2026 – ARC-AGI-3. https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3, 2026. Kaggle code competition; public leaderboard snapshot 2026-10-25.

Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Rémi Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, 2016.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

Jeroen Cottaar and Tufa Labs. The duck: ARC-AGI-3 inference harness (milestone 1 winning solution). https://github.com/Tufalabs/duck-harness, 2026. Commit 7652836; Kaggle notebook jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner.

Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.

Daniel Franzen. ARC-AGI-3 milestone 2 solution. https://www.kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution, 2026. Code: https://github.com/da-fr/arc-agi-3-solution (Apache-2.0); write-up: https://www.kaggle.com/writeups/dfranzen/arc-agi-3-milestone-2.

heliosli. Try merging it, you might get lucky and win gold (stagnation restart). https://www.kaggle.com/code/heliosli/try-merging-it-you-might-get-lucky-and-win-gold, 2026. Kaggle notebook, V4.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022.

leoprovorov. Fable / ASTRA play: ARC-3 handbook harness (F001 fable layer). https://www.kaggle.com/code/leoprovorov/fable-astra-play-arc-3-handbook-harness, 2026. Kaggle notebook and dataset, Apache-2.0.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

OpenAI. How two settings tripled our ARC-AGI-3 scores. https://openai.com/index/how-two-settings-tripled-our-arc-agi-3-scores/, 2026. 29 July 2026.

Qwen Team. Qwen3.8-Flash-Next, 2026. Used as Intel AutoRound W4A16 quantisation (Kaggle model dfranzen/intel-qwen3.8-flash-next-w4a16-autoround) with Albucino's MTP draft model; Qwen Community License 1.0.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

shiiin9 and Affectify ARC, Osaka Metropolitan University. Affectify ARC 31.54 in a single sub (D′ slot priority). https://www.kaggle.com/code/shiiin9/affectify-arc-31-54-in-a-single-sub, 2026. Kaggle notebook.

skarin. ARC-AGI-3 27.80 LB – 100-trajectory audit. https://www.kaggle.com/code/skarin/arc-agi-3-27-80-lb-100-trajectory-audit, 2026. Kaggle notebook; source of the 100 Franzen v3 trajectories we re-analysed.

Dries Smit. Stochastic goose: ARC-AGI-3 developer preview agent competition solution. https://github.com/DriesSmit/ARC3-solution, 2025. Tufa Labs.

William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.

Supplementary Materials

The following content was not necessarily subject to peer review.

9 Hyperparameters, compute and all versions

Table 2: Compute. Training on one Kaggle T4 per seed (seeds in parallel); evaluation on Kaggle CPU, 1 thread per job, for 3 eval seeds × 5 dev games × 500 actions (per training seed). Test-time TD learning makes DQN evaluation ∼10× slower than PPO's. Total training: 10.45 T4-hours over 21 runs (incl. the T23 effect-model diagnostic).

Version	Train steps	Train h/seed (T4)	Eval s/train seed (CPU)	Train lv (s0/s1/s2)
TEAM-10 DQN	400,005	0.30–0.32	1,166	7/4/8
TEAM-11 +novelty	400,005	0.43–0.43	1,066	5/5/6
TEAM-12 +effect mask	400,005	0.49–0.50	1,350	4/6/4
TEAM-20 PPO	401,280	0.37–0.37	110	3/5/5
TEAM-21 +RND	401,280	0.45–0.46	68	5/7/6
TEAM-22 +LSTM	401,280	0.93–0.94	109	5/5/8
TEAM-30 graph+online stats	0	–	89 (9 seeds)	–
TEAM-37 +inverse actions	0 (+T21 prior)	–	92	–
TEAM-45 component clicks	0 (+T21 prior)	–	123	–

Table 3: Hyperparameters (identical across seeds; all learned versions: 400k steps on 15 games, episode cap 500; configs in `experiments/train/*/config.json`).

DQN (T10–T12)	replay 100k, batch 32, Adam 10^{-4}, γ 0.99, Huber, train every 4 steps, start 20k, hard target sync 10k, ϵ 1.0→0.05 over 200k; eval: ϵ 0.05, 1 TD update/action (batch 32, lr 10^{-4}, target sync 100)
T11 / T12	novelty $\beta = 0.05$ (exact frame counts) / effect head BCE weight 1.0, mask threshold $p < 0.1$
PPO (T20–T22)	MaskablePPO, n_{steps} 128 × 15 envs, batch 240, 4 epochs, Adam $2.5 \cdot 10^{-4}$, γ 0.99, GAE λ 0.95, clip 0.2, entropy 0.01
T21 / T22	RND coefficient 0.05, EMA(α=0.01)-normalised / + LSTM 256 (RecurrentPPO, masks as input channel)
Graph (T30–T47)	Beta(1,1)-Thompson effect stats; prior weight 0.3 (T31 +); prune after 3 failed tries; inverse pair after 2 undos, 0 counter-ex.; ticker row: ≥10 changing steps, changes in ≥50%, median ≤2 px, ≥3 actions; components ≤96 per state

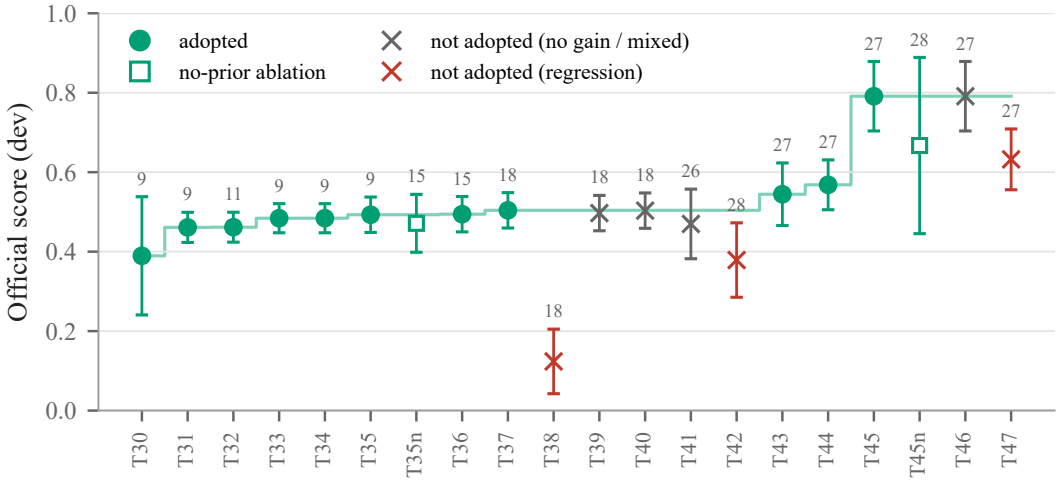


Figure 7: Graph-explorer progression on dev (mean ± SD over training seeds; numbers = levels). The line is the best adopted version so far. Red = regressions (T38 patch-keyed stats, T42 small objects only, T47 size prior).

Table 4: Every evaluated version (same columns as Table 1). G1 = graph exploration with the pretrained T12 effect head.

Ver.	Change	Dev score	Lv	Level 1 on dev: runs @ actions	Train	Held-out	Kaggle
Rand.		0.0004 ± 0.0006	1	lp85 1/3 @352	0.0111	–	0.02†
T10	DQN	0	0	–	–	–	0.00
T10f	DQN frozen	0	0	–	–	–	–
T11	+novelty	0	0	–	–	–	0.00
T11f	frozen	0	0	–	–	–	–
T12	+effect mask	0.0044 ± 0.0063	3	lp85 3/9 @101–120	–	–	0.00
T12f	frozen	0.0036 ± 0.0033	6	lp85 6/9 @116–307	–	–	–
T20	PPO	0.0004 ± 0.0006	1	lp85 1/9 @212	–	–	0.00
T21	+RND	0.0247 ± 0.0317	5	lp85 4/9 @124–489; ls20 1/9 @41	–	–	0.05
T22	+LSTM	0.0003 ± 0.0004	1	lp85 1/9 @241	–	–	–
GRAPH-v1	(DQN effect head)	0.0013 ± 0.0010	4	lp85 4/9 @149–380	–	–	–
T30	graph+online stats	0.389 ± 0.149	9	lp85 9/9 @16–72	0.0578	–	–
T31	+PPO prior	0.461 ± 0.038	9	lp85 9/9 @12–24	–	–	–
T32	+ticker mask	0.462 ± 0.038	11	lp85 9/9 @12–24; ls20 2/9 @413–416	–	–	–
T33	+prune	0.484 ± 0.037	9	lp85 9/9 @12–20	–	–	–
T34	+timer deaths	0.484 ± 0.037	9	lp85 9/9 @12–20	–	–	–
T35	+click equivalence	0.493 ± 0.045	9	lp85 9/9 @15–25	–	–	–
T35n	no prior	0.471 ± 0.073	15	lp85 9/9 @15–21; ls20 6/9 @297–473	0.121	–	–
T36	prior on clicks	0.494 ± 0.045	15	lp85 9/9 @15–25; ls20 6/9 @367–473	–	–	–
T37	+inverse actions	0.504 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	0.07
T38	patch stats	0.124 ± 0.081	18	lp85 9/9 @17–220; ls20 9/9 @128–376	–	–	–
T39	value frontier	0.497 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @205–421	–	–	–
T40	relax when stuck	0.503 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @128–458	–	–	–
T41	+object clicks	0.470 ± 0.088	26	lf52 9/9 @34–95; lp85 9/9 @17–36; ls20 6/9 @240–413	–	–	–
T42	small objects	0.379 ± 0.094	28	ka59 1/9 @256; lf52 9/9 @35–99; lp85 9/9 @17–33; ls20 9/9 @128–287	–	–	–
T43	objects as fallback	0.545 ± 0.079	27	lf52 9/9 @52–349; lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	–
T44	object promotion	0.568 ± 0.063	27	lf52 9/9 @46–295; lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	–
T45	component clicks	0.791 ± 0.087	27	lf52 9/9 @24–56; lp85 9/9 @14–30; ls20 9/9 @128–287	–	–	–
T45n	no prior	0.667 ± 0.222	28	lf52 9/9 @23–59; lp85 9/9 @11–33; ls20 8/9 @128–374	0.0811	–	–
T46	centres fallback	0.791 ± 0.087	27	lf52 9/9 @24–56; lp85 9/9 @14–30; ls20 9/9 @128–287	–	–	–
T47	size prior	0.632 ± 0.077	27	lf52 9/9 @13–45; lp85 9/9 @16–49; ls20 9/9 @128–287	–	–	–

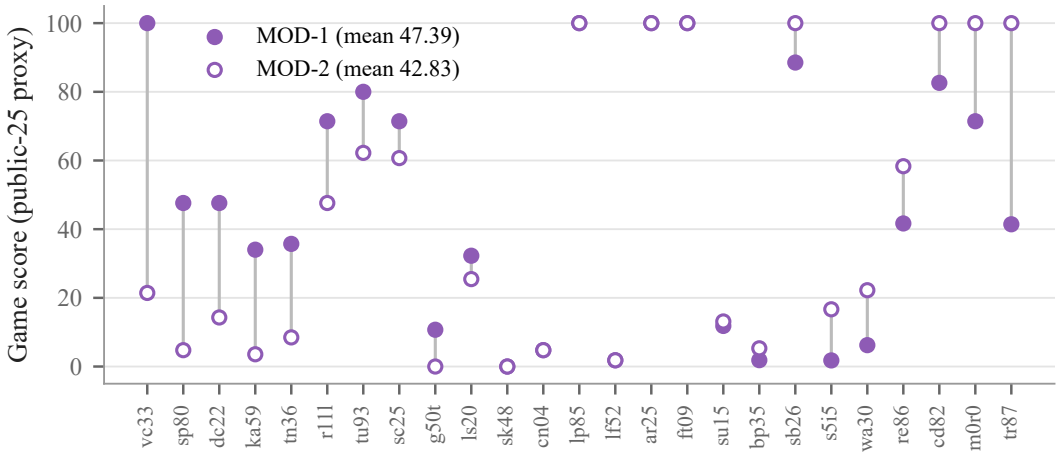


Figure 8: Matched one-pass proxy on the 25 public games (RTX PRO 6000, ≈2 h each, launched together): MOD-1 47.39 vs. MOD-2 42.83. Per-game swings of up to 79 points in both directions show why one pass resolves only large effects.

10 Kaggle submissions (evidence)

All 16 submissions with IDs, script versions and output sizes are in `experiments/kaggle_submissions.csv`. Nominated: submission **57556604**, notebook `thendonelufule/arc3-m2-modf-sub`, script version 359469291, public score **29.15**, 2026-10-23 09:42:37 UTC.